Optimal two-stage stratified cluster allocation with gaSampling

Mokwenda & Campher

2026-10-22

1. Overview

`gaSampling` finds an allocation of clusters and secondary sampling units (SSUs) for a two-stage stratified cluster design. It uses a genetic algorithm (GA) to minimise the variance of the mean estimator, subject to a minimum total sample size and a budget.

The design has three levels:

- **strata**, here the nine provinces of South Africa,
- **clusters**, the primary sampling units, here wards,
- **SSUs**, here early childhood development (ECD) centres.

The workflow in this vignette is:

1. prepare the stratum-level input from SSU-level data,
2. construct a feasible starting allocation,
3. run the GA to obtain the allocation,
4. check the result.

The method follows Cochran (1977, Chapter 10); the GA is built on the `GA` package (Scrucca, 2013).

2. Data preparation

The package includes `ecd_subset`, a small extract of the 2021 ECD census. It contains all ECD centres in a random sample of up to 40 wards per province, with at least two centres per ward. It is intended for illustration only and should not be used for inference about the full census. The response is the number of children present at each centre (`count_children_present`).

```
data("ecd_subset")
str(ecd_subset)
#> 'data.frame':    2823 obs. of  12 variables:
#>  $ id                    : num  1e+06 1e+06 1e+06 1e+06 1e+06 ...
#>  $ province              : chr  "Western Cape" "Western Cape" "Western Cape" "Western Cape" ...
#>  $ id_prov               : chr  "WC" "WC" "WC" "WC" ...
#>  $ id_dc                 : chr  "DC1" "DC1" "DC1" "DC1" ...
#>  $ dc                    : chr  "West Coast" "West Coast" "West Coast" "West Coast" ...
#>  $ id_mn                 : chr  "WC011" "WC011" "WC011" "WC014" ...
#>  $ mn                    : chr  "Matzikama" "Matzikama" "Matzikama" "Saldanha Bay" ...
#>  $ id_ward               : num  10101006 10101006 10101006 10104004 10104004 ...
#>  $ count_children_present: num  22 35 28 32 25 37 38 5 13 30 ...
#>  $ capacity              : num  50 107 NA NA 50 NA 107 NA NA NA ...
#>  $ count_staff_all       : num  7 10 5 NA 4 6 8 1 3 7 ...
#>  $ fees_amount           : chr  "40" "100" "600" NA ...
```

```
wards   <- tapply(ecd_subset$id_ward, ecd_subset$province,
                  function(x) length(unique(x)))
centres <- table(ecd_subset$province)
data.frame(province = names(wards),
           wards    = as.vector(wards),
           centres  = as.vector(centres))
#>        province wards centres
#> 1  Eastern Cape    39     281
#> 2    Free State    37     237
#> 3       Gauteng    40     607
#> 4 KwaZulu-Natal    38     339
#> 5       Limpopo    40     374
#> 6    Mpumalanga    39     265
#> 7    North West    37     206
#> 8 Northern Cape    31     113
#> 9  Western Cape    38     401
```

`read_survey_data()` takes a data frame with one row per SSU and the names of the stratum, cluster and response columns. It cleans identifiers, checks that clusters nest within strata, coerces the response to numeric (missing values are handled according to `na_action`) and warns about negative responses and single-SSU clusters.

```
survey_data <- read_survey_data(
  data       = ecd_subset,
  stratum_id = "province",
  cluster_id = "id_ward",
  y_var      = "count_children_present",
  na_action  = "warn"
)
#>
#> -- gasampling: survey data loaded --
#>
#>   Rows (SSUs)    : 2823
#>   Strata         : 9
#>   Clusters       : 339
#>   Analysis var   : count_children_present
#>   y range        : 2.00 -- 184.00
#>   Missing in y   : 0
#>
#>   Standardised column names: strata | cluster | count_children_present
```

3. Starting allocation

Before building the stratum table we need the total number of clusters to consider. `frame_c()` returns the smaller of the number of available clusters (distinct stratum and cluster pairs in the data) and the number we can afford once the cost of the required number of SSUs is reserved, namely `floor((max_cost - unit_cost * total_n) / cluster_cost)`. It is a preliminary total across all strata, not an allocation.

The sample-size target, budget and costs are set first. We ask for a total sample of 250 centres, a budget of R100,000, and costs of R1,000 per sampled ward and R100 per sampled centre.

```
target_n     <- 250
max_cost     <- 100000
```

```r
cost_cluster <- 1000
cost_unit    <- 100

total_c <- frame_c(
  survey_data  = survey_data,
  max_cost     = max_cost,
  unit_cost    = cost_unit,
  cluster_cost = cost_cluster,
  total_n      = target_n
)
total_c
#> [1] 75
```

`prepare_sampling_frame()` aggregates the cleaned data to the stratum level. For each stratum it returns the number of clusters C_h, the number of SSUs N_h, the weight W_h, the between- and within-cluster variance components σ^2_{bh} and σ^2_{wh}, and the cluster and unit costs. The stratum table is in `$strata`.

```r
frame <- prepare_sampling_frame(
  data         = survey_data,
  y_var        = "count_children_present",
  total_c      = total_c,
  cost_cluster = cost_cluster,
  cost_unit    = cost_unit
)
#>
#> -- gasampling: sampling frame prepared --
#>
#>   Strata (H)        : 9
#>   Total clusters    : 339
#>   Total SSUs        : 2823
#>   Suggested c_h     : 9 8 9 8 9 9 8 7 8  (total = 75)
#>
#>   Stratum summary:
#>   Stratum                C_h      N_h      W_h  sigma2_bh  sigma2_wh
#>   ------------------------------------------------------------------------
#>   Eastern Cape            39      281   0.0995   129.9720   193.1454
#>   Free State              37      237   0.0840   236.8972   658.0395
#>   Gauteng                 40      607   0.2150    95.7317   631.9897
#>   KwaZulu-Natal           38      339   0.1201   105.3761   390.3766
#>   Limpopo                 40      374   0.1325   151.6445   515.5074
#>   Mpumalanga              39      265   0.0939   214.2332   661.2882
#>   North West              37      206   0.0730   165.3777   403.2032
#>   Northern Cape           31      113   0.0400   141.8304   266.8869
#>   Western Cape            38      401   0.1420   236.2862   669.4901
strata <- frame$strata
# The cluster and unit costs are the same in every stratum, so they are omitted here
strata[, c("stratum", "C_h", "N_h", "W_h", "sigma2_bh", "sigma2_wh")]
#>          stratum C_h N_h        W_h sigma2_bh sigma2_wh
#> 1  Eastern Cape  39 281 0.09953950 129.97199  193.1454
#> 2    Free State  37 237 0.08395324 236.89725  658.0395
#> 3       Gauteng  40 607 0.21501948  95.73173  631.9897
#> 4 KwaZulu-Natal  38 339 0.12008502 105.37610  390.3766
#> 5       Limpopo  40 374 0.13248317 151.64451  515.5074
#> 6    Mpumalanga  39 265 0.09387177 214.23322  661.2882
```

```
#> 7    North West  37 206 0.07297202 165.37774  403.2032
#> 8 Northern Cape  31 113 0.04002834 141.83042  266.8869
#> 9  Western Cape  38 401 0.14204747 236.28616  669.4901
```

Three helper functions support the construction of a starting allocation:

- `allocate_ch_by_weight()` distributes a total number of clusters across strata using relative weights (stratum size, variability or equal weights), subject to lower and upper bounds.
- `check_total_n_feasible()` checks whether fixed cluster counts and bounds on units per cluster permit the minimum sample size to be met. It does not check budget feasibility.
- `find_feasible_joint_start()` tries several starting rules, repairs each resulting allocation, checks feasibility and returns a starting suggestion with diagnostics. It stops with an error if no feasible start is found.

Here we use the Neyman-proportional starting rule.

```
start <- find_feasible_joint_start(
  strata                      = strata,
  total_n                     = target_n,
  max_cost                    = max_cost,
  max_candidate_c             = total_c,
  default_starting_suggestion = "3. Neyman proportional",
  require_all_starts          = FALSE
)
start$total_c
#> [1] 75
start$diagnostics
#>                     starting_suggestion repaired_total_n repaired_total_c
#> 1       1. Proportional to stratum size              253               75
#> 2 2. Proportional to between-cluster SD              255               75
#> 3                3. Neyman proportional              253               75
#> 4               4. Equal c_h allocation              255               75
#>   repaired_total_cost sample_size_gap repair_iterations repaired_exactly
#> 1              100300               3                 4             TRUE
#> 2              100500               5                 3             TRUE
#> 3              100300               3                 3             TRUE
#> 4              100500               5                 3             TRUE
#>   feasible
#> 1     TRUE
#> 2     TRUE
#> 3     TRUE
#> 4     TRUE
```

4. Running the genetic algorithm

`allocate_twostage_ga_joint()` searches over the cluster counts c_h and the unit counts k_h together. Candidate solutions are real-valued, then rounded and repaired during fitness evaluation, and the selected solution is repaired and checked again before it is returned. Passing `suggestion = start$suggestion` seeds the search with the starting allocation; if it is omitted, the GA builds its own simple starting point.

The settings below are deliberately small so the vignette builds quickly. For real work use a larger population and more iterations (for example `popSize = 100`, `maxiter = 500`, `run = 30`).

```
res <- allocate_twostage_ga_joint(
  strata     = strata,
  total_n    = target_n,
  max_cost   = max_cost,
  popSize    = 30,
  maxiter    = 50,
  run        = 15,
  seed       = 2026,
  suggestion = start$suggestion
)
res$allocation
#>         stratum c_h k_h n_h var_between var_within var_total
#> 1  Eastern Cape   7   3  21  0.18396814 0.09112887 0.2750970
#> 2    Free State  11   3  33  0.15178969 0.14054422 0.2923339
#> 3       Gauteng  11   3  33  0.40236375 0.88542485 1.2877886
#> 4 KwaZulu-Natal   7   3  21  0.21708095 0.26806622 0.4851472
#> 5       Limpopo   9   3  27  0.29573698 0.33511404 0.6308510
#> 6    Mpumalanga   8   3  24  0.23597545 0.24280046 0.4787759
#> 7    North West   6   3  18  0.14677040 0.11927904 0.2660494
#> 8 Northern Cape   4   4  16  0.05681258 0.02672652 0.0835391
#> 9  Western Cape  12   5  60  0.39730500 0.22514376 0.6224488
```

The returned object contains the feasibility diagnostics, the total variance achieved, the between- and within-cluster variance contributions, the total sample size, the total cost, the allocation by stratum and the fitted GA object.

The procedure is heuristic and stochastic, so it does not guarantee a globally optimal or even a feasible allocation. **Always inspect the diagnostics before using the allocation.**

```
res$feasible
#> [1] TRUE
res$variance
#> [1] 4.422031
res$total_n
#> [1] 253
res$total_cost
#> [1] 100300
sum(res$allocation$c_h)   # total clusters (wards)
#> [1] 75
```

5. Checking an allocation

`check_constraints()` evaluates an allocation without modifying it. It checks that cluster and unit counts are integer valued and within their bounds, that the achieved sample size meets the minimum (`total_n` is a minimum, not an exact target), and, when `max_cost` is supplied, that the total cost lies between `max_cost * min_budget_use` and `max_cost * (1 + budget_tol)`. The defaults are `min_budget_use = 0.95` and `budget_tol = 0.02`, so spending must fall between 95% and 102% of the reference budget.

It returns an overall indicator `feasible` and separate indicators for the integer, cluster, unit, sample-size and cost checks, together with the achieved `total_n`, `total_cost` and the stratum sample sizes `n_h`. The bounds above are the ones used inside the search: at least two clusters per stratum, at most `C_h` clusters, and between 1 and `total_n` units per cluster.

```r
check_constraints(
  c_h            = res$allocation$c_h,
  k_h            = res$allocation$k_h,
  total_n        = target_n,
  max_cost       = max_cost,
  cost_cluster   = strata$cost_cluster,
  cost_unit      = strata$cost_unit,
  min_clusters   = 2,
  min_units      = 1,
  max_clusters   = strata$C_h,
  max_units      = rep(target_n, nrow(strata)),
  min_budget_use = 0.95,
  budget_tol     = 0.02
)
#> $feasible
#> [1] TRUE
#>
#> $integer_ok
#> [1] TRUE
#>
#> $cluster_ok
#> [1] TRUE
#>
#> $unit_ok
#> [1] TRUE
#>
#> $sample_size_ok
#> [1] TRUE
#>
#> $cost_ok
#> [1] TRUE
#>
#> $total_n
#> [1] 253
#>
#> $total_cost
#> [1] 100300
#>
#> $n_h
#> [1] 21 33 33 21 27 24 18 16 60
```

The two objective components can also be computed directly for any allocation:

```r
variance_twostage(
  c_h = res$allocation$c_h, k_h = res$allocation$k_h,
  W_h = strata$W_h, sigma2_bh = strata$sigma2_bh, sigma2_wh = strata$sigma2_wh
)
#> [1] 4.422031

cost_twostage(
  c_h = res$allocation$c_h, k_h = res$allocation$k_h,
  cost_cluster = strata$cost_cluster, cost_unit = strata$cost_unit
```

```
)
#> [1] 100300
```

`variance_twostage()` does not apply finite population corrections and does not check feasibility. `cost_twostage()` returns the total in the currency units you supplied; budget feasibility is assessed separately by `check_constraints()`.

6. Repair and fitness (for reference)

Two functions work behind the scenes inside the GA:

- `repair_joint()` adjusts a candidate so that it satisfies the minimum sample size, the allocation bounds and any budget constraints. It is not guaranteed to produce a feasible allocation, so its diagnostics must be examined.
- `fitness_ckh_constrained()` scores a candidate. A feasible repaired allocation scores minus its variance, and an infeasible one receives a large penalty, because the GA maximises fitness.

See their help pages for the full argument lists.

References

Cochran, W. G. (1977). *Sampling Techniques* (3rd ed.). Wiley.

Scrucca, L. (2013). GA: A package for genetic algorithms in R. *Journal of Statistical Software*, 53(4), 1-37.